

ShopEZ -Full Stack E-Commerce Platform

Student Details:

Student Name: Narendula Jashwanth

Domain: Web Development

Technology Stack: MERN Stack

1.Introduction:

ShopEZ is a full-stack e-commerce web application developed using the MERN stack. The project is designed to provide users with a secure, convenient, and user-friendly online shopping experience. It allows customers to browse products, search and filter items, manage wishlists and carts, place orders, make online payments, and track their purchases. The project also includes an administrator module that enables product management, order status updates, and platform-level control. The application integrates modern web development technologies such as React.js, Node.js, Express.js, MongoDB, JWT authentication, Razorpay payment gateway, Cloudinary image storage, and Nodemailer email services. The project was fully developed as an individual project to demonstrate practical knowledge of full stack development, database management, API integration, authentication, payment processing, and deployment of a real-world application.

2.Project Overview:

Purpose:

The main purpose of ShopEZ is a full-stack e-commerce web application developed using the MERN stack. The main purpose of this project is to provide users with a smooth online shopping experience where they can browse products, view details, add items to cart or wishlist, place orders, make payments, and track their purchase history.

The application also includes an admin panel where administrators can manage products, orders, users, banners, categories, and business analytics.

Goals of the project:

The main goals of the project are:

- To develop a complete real-world e-commerce platform.
- To implement secure user authentication and role-based access.
- To provide product browsing, cart, wishlist, order, and payment features.
- To create an admin dashboard for product and order management.
- To integrate online payment using Razorpay.

- To design a responsive and user-friendly interface.
- To deploy the frontend and backend for public access.

Key Features

1.User Features

- User registration and login.
- Secure JWT-based authentication.
- Product listing with search, category filter, and sorting.
- Product details page with images, price, description, stock, and ratings.
- Add to cart, update quantity, and remove cart items.
- Wishlist management.
- Checkout with address details.
- Cash on Delivery and Razorpay payment support.
- Order history and order status view.
- Product review and rating system.
- Profile management.
- Location detection for saving user address.

2.Admin Features

- Admin login and protected dashboard.
- Add, update, and delete products.
- Manage product images using Cloudinary.
- Manage order status.
- View users, orders, revenue, and analytics.
- Manage banners, categories, coupons, and promotions.
- View recent orders and pending orders.

Project links:

Frontend:vip-c2-jashwanth.vercel.app

Backend :<https://vip-c2-jashwanth.onrender.com>

GitHub:<https://github.com/narendulajashwanth007-hash/VIP-C2-Jashwanth.git>

3.Architecture:

ShopEZ follows a client-server architecture using the MERN stack.

Frontend Architecture

The frontend is built using React.js with Vite. It provides a responsive user interface for customers and administrators.

Main Frontend Responsibilities

- Display product data received from backend APIs.
- Manage user navigation using React Router.
- Handle login, register, cart, wishlist, checkout, and order pages.
- Display admin dashboard and product management pages.
- Show success, error, loading, and empty states.
- Communicate with backend APIs using Axios.
- Store authentication state and user role.
- Provide a responsive UI using Tailwind CSS.

Backend Architecture

The backend is developed using Node.js and Express.js. It acts as the main server that handles API requests, authentication, business logic, and database operations.

Main Backend Responsibilities

- Handle user registration and login.
- Generate and verify JWT tokens.
- Protect private routes using authentication middleware.
- Provide APIs for products, cart, wishlist, orders, reviews, payments, and admin analytics.
- Validate user roles for admin-only routes.
- Process Razorpay payment verification.
- Send email notifications using Nodemailer.
- Handle errors using centralized error middleware.

Database Architecture

MongoDB is used as the database. Mongoose is used to define schemas and interact with MongoDB.

The Major database Collections include:

- Users
- Products
- Cart
- Orders
- Wishlists
- Reviews
- Admin-related records

Stores user details, authentication data, role, and address, Stores product details, price, stock, category, images, and variants, Stores products added to the user cart, Stores user wishlist items, Stores order details, payment status, delivery status, and address, Stores product reviews and ratings, Stores discount coupon data, Stores homepage banner details, Stores product category details.

4.Setup Instructions:

A.Prerequisites

Before running the project, the following software and tools are required:

- Node.js
- npm
- MongoDB Atlas or MongoDB Community Server
- Git
- Code editor such as Visual Studio Code
- Razorpay account for payment integration
- Cloudinary account for image upload
- Email account or SMTP service for Nodemailer

B.Repository Link:

<https://github.com/narendulajashwanth007-hash/VIP-C2-Jashwanth.git>

C.Installation Steps:

Step 1: Clone the Repository

git clone https: <https://github.com/narendulajashwanth007-hash/VIP-C2-Jashwanth.git>

Step 2: Open the Project Folder

```
cd SHOPEZ-E-commerce-Application
```

Step 3: Install Frontend Dependencies

```
cd client
```

```
npm install
```

Step 4: Install Backend Dependencies

```
cd ../server
```

```
npm install
```

D.Environment Variables:

Create a .env file inside the server folder and add the required environment variables.

Example:

```
PORT=5000
```

```
MONGO_URI=your_mongodb_connection_string
```

```
JWT_SECRET=your_jwt_secret_key
```

```
JWT_EXPIRES_IN=7d
```

```
RAZORPAY_KEY_ID=your_razorpay_key_id
```

```
RAZORPAY_KEY_SECRET=your_razorpay_secret
```

```
CLOUDINARY_CLOUD_NAME=your_cloudinary_cloud_name
```

```
CLOUDINARY_API_KEY=your_cloudinary_api_key
```

```
CLOUDINARY_API_SECRET=your_cloudinary_api_secret
```

```
EMAIL_USER=your_email
```

```
EMAIL_PASS=your_email_password
```

```
CLIENT_URL=http://localhost:5173
```

5.Folder Structure:

Client Folder Structure(frontend)

client/

```
|
|
|— public/
|
|— src/
|   |— assets/
|   |— components/
|   |— pages/
|   |— context/
|   |— services/
|   |— routes/
|   |— utils/
|   |— App.jsx
|   └─ main.jsx
|
|— package.json
└─ vite.config.js
```

Description

- **assets:** Stores images and static files.
- **components:** Contains reusable UI components like Navbar, Footer, ProductCard, Loader, and ProtectedRoute.
- **pages:** Contains main pages such as Home, Products, ProductDetails, Cart, Wishlist, Checkout, Orders, Login, Register, and Admin Dashboard.
- **context:** Manages global state such as authentication and cart data.
- **services:** Contains API calling functions using Axios.
- **routes:** Handles protected and role-based routing.
- **utils:** Contains helper functions.
- **App.jsx:** Main application component.

- **main.jsx:** React entry point.

Server Folder Structure(backend)

server/

```
|
|
├── config/
├── controllers/
├── middleware/
├── models/
├── routes/
├── utils/
├── uploads/
├── server.js
├── package.json
└── .env
```

Description

- **config:** Contains database and cloud service configuration.
- **controllers:** Contains business logic for users, products, cart, wishlist, orders, reviews, payments, and admin.
- **middleware:** Contains authentication, authorization, upload, and error handling middleware.
- **models:** Contains Mongoose schemas.
- **routes:** Contains API route definitions.
- **utils:** Contains reusable helper functions.
- **uploads:** Temporarily stores uploaded files if required.
- **server.js:** Main backend entry point.

6. Running the Application:

Run Backend

cd server

npm start

Backend runs on:

<http://localhost:5000>

Run Frontend

cd client

npm run dev

Frontend runs on:

<http://localhost:5173>

Deployed Links

Frontend:

vip-c2-jashwanth.vercel.app

Backend:

<https://vip-c2-jashwanth.onrender.com>

GitHub

<https://github.com/narendulajashwanth007-hash/VIP-C2-Jashwanth.git>

7.API Documentation:

Authentication APIs:

Method	Endpoint	Description
POST	<code>/api/auth/register</code>	Register a new user
POST	<code>/api/auth/login</code>	Login user
GET	<code>/api/auth/profile</code>	Get logged-in user profile
PUT	<code>/api/auth/profile</code>	Update user profile
POST	<code>/api/auth/logout</code>	Logout user

Example Login Request

```
{  
  "email": "user@example.com",
```



```
"password": "password123"

}
```

Example Login Response

```
{
  "success": true,
  "message": "Login successful",
  "token": "jwt_token_here",
  "user": {
    "name": "User",
    "email": "user@example.com",
    "role": "user"
  }
}
```

Product APIs:

Method	Method Endpoint Description	Method Endpoint Description
GET	/api/products	Get all products
GET	/api/products/:id	Get single product details
POST	/api/products	Add new product - Admin only
PUT	/api/products/:id	Update product - Admin only
DELETE	/api/products/:id	Delete product - Admin only
GET	/api/products?category=categoryName	Filter products by category
GET	/api/products?search=keyword	Search products

Cart APIs:

Method	Endpoint	Description
GET	/api/cart	Get user cart
POST	/api/cart	Add product to cart

PUT	/api/cart/:itemId	Update cart item quantity
DELETE	/api/cart/:itemId	Remove item from cart
DELETE	/api/cart	Clear cart

Wishlist APIs:

Method	Endpoint	Description
GET	/api/wishlist	Get user wishlist
POST	/api/wishlist	Add product to wishlist
DELETE	/api/wishlist/:productId	Remove product from wishlist

Order APIs:

Method	Method Endpoint Description	Method Endpoint Description
POST	/api/orders	Place new order
GET	/api/orders/my-orders	Get user orders
GET	/api/orders/:id	Get order details
PATCH	/api/orders/:id/cancel	Cancel order before shipping
PATCH	/api/orders/:id/status	Update order status - Admin only

Payment APIs:

Method	Method Endpoint Description	Method Endpoint Description
POST	/api/payment/create-order	Create Razorpay order
POST	/api/payment/verify	Verify Razorpay payment

Review APIs:

Method	Endpoint	Description
POST	/api/reviews/:productId	Add product review
GET	/api/reviews/:productId	Get product reviews

Admin APIs:

Method	Method Endpoint Description	Method Endpoint Description
GET	/api/admin/analytics	Get admin dashboard analytics
GET	/api/admin/users	Get all users
GET	/api/admin/orders	Get all orders
PATCH	/api/admin/orders/:id/status	Update order status
POST	/api/admin/banners	Add banner
POST	/api/admin/categories	Add category
POST	/api/admin/coupons	Add coupon

Authentication:

Authentication in ShopEZ is handled using JWT tokens and bcrypt password hashing.

Authentication Flow

1. The user registers with name, email, and password.
2. The password is hashed using bcrypt before storing it in MongoDB.
3. During login, the entered password is compared with the hashed password.
4. If the credentials are valid, a JWT token is generated.
5. The token is used to access protected routes.

6. Middleware verifies the token before allowing access to private APIs.

8.Authorization:

Role-based authorization is implemented using user roles.

Roles

Role	Access
User	Product browsing, cart, wishlist, checkout, orders, reviews
Admin	Product management, order management, analytics, users, coupons, banners

Security Features:

- Password hashing using bcrypt.
- JWT-based protected routes.
- Admin-only route protection.
- Input validation.
- Centralized error handling.
- Secure environment variables.
- Payment verification before marking orders as paid

9.User Interface:

The user interface is designed to be clean, responsive, and easy to use. Tailwind CSS is used for styling.

Main UI Pages

a.Home Page

The home page displays banners, featured products, categories, and navigation options for users.

b.Product Listing Page

The product listing page allows users to view all products, search products, filter by category, and sort based on price or relevance.

c.Product Details Page

The product details page displays product images, price, stock, description, ratings, reviews, and add-to-cart option.

d.Cart Page

The cart page allows users to view selected products, update quantity, remove items, and proceed to checkout.

e.Wishlist Page

The wishlist page allows users to save products for later purchase.

Checkout Page

The checkout page collects delivery address, payment option, and order confirmation details.

f.Orders Page

The orders page displays user order history and current order status.

Admin Dashboard

The admin dashboard displays analytics such as total users, total products, total orders, revenue, pending orders, delivered orders, and recent orders.

g.Admin Product Management

The admin can add, update, and delete products along with product images, stock, price, category, and description.

10.Testing:

Testing was performed to ensure that all major features of the application work correctly.

Testing Strategy

The project was tested using manual testing, API testing, and UI flow testing.

Testing Tools

- Browser developer tools
- Postman or REST client
- MongoDB Compass

- Console logs for debugging
- Manual UI testing

Important Test Cases

Remove item from cart	Product should be removed	Passed
Add product to wishlist	Product should appear in wishlist	Passed
Place COD order	Order should be created with pending payment	Passed
Razorpay payment	Payment should be verified successfully	Passed
View order history	User orders should be displayed	Passed
Add product review	Review and rating should be saved	Passed
Admin adds product	Product should appear in product list	Passed
Admin updates product	Updated details should be saved	Passed
Admin deletes product	Product should be removed	Passed
Update cart quantity	Order status should change	Passed
Remove item from cart	Product should be removed	Passed
Add product to wishlist	Product should appear in wishlist	Passed
Place COD order	Order should be created with pending payment	Passed
Razorpay payment	Payment should be verified successfully	Passed
View order history	User orders should be displayed	Passed
Add product review	Review and rating should be saved	Passed
Admin adds product	Product should appear in product list	Passed

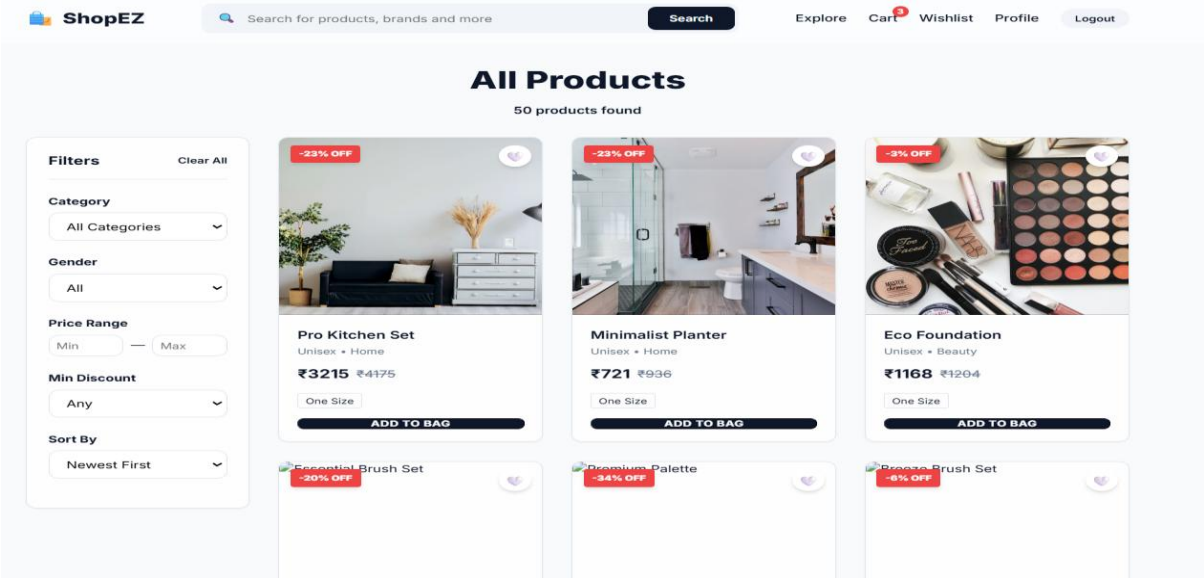
Admin updates product	Updated details should be saved	Passed
-----------------------	---------------------------------	--------

11.Screenshots or Demo:

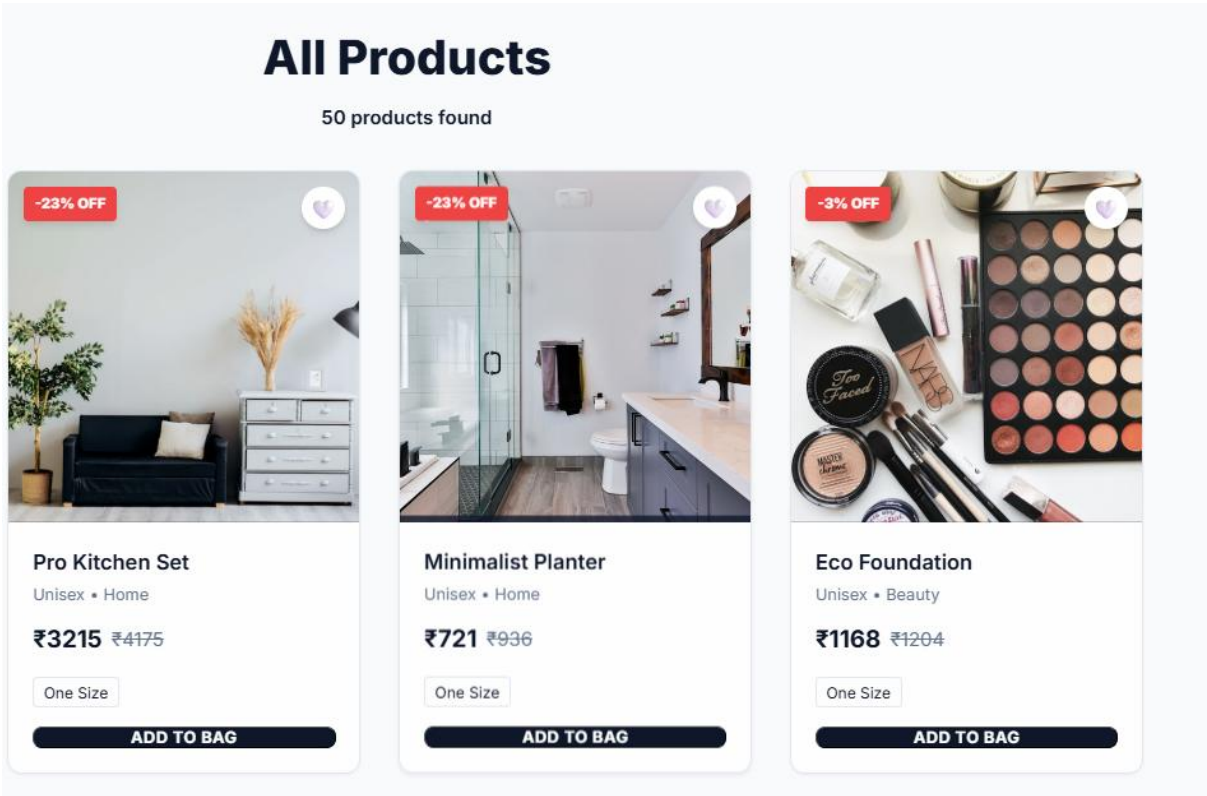
Screenshots to Include:

The following screenshots can be added to the report:

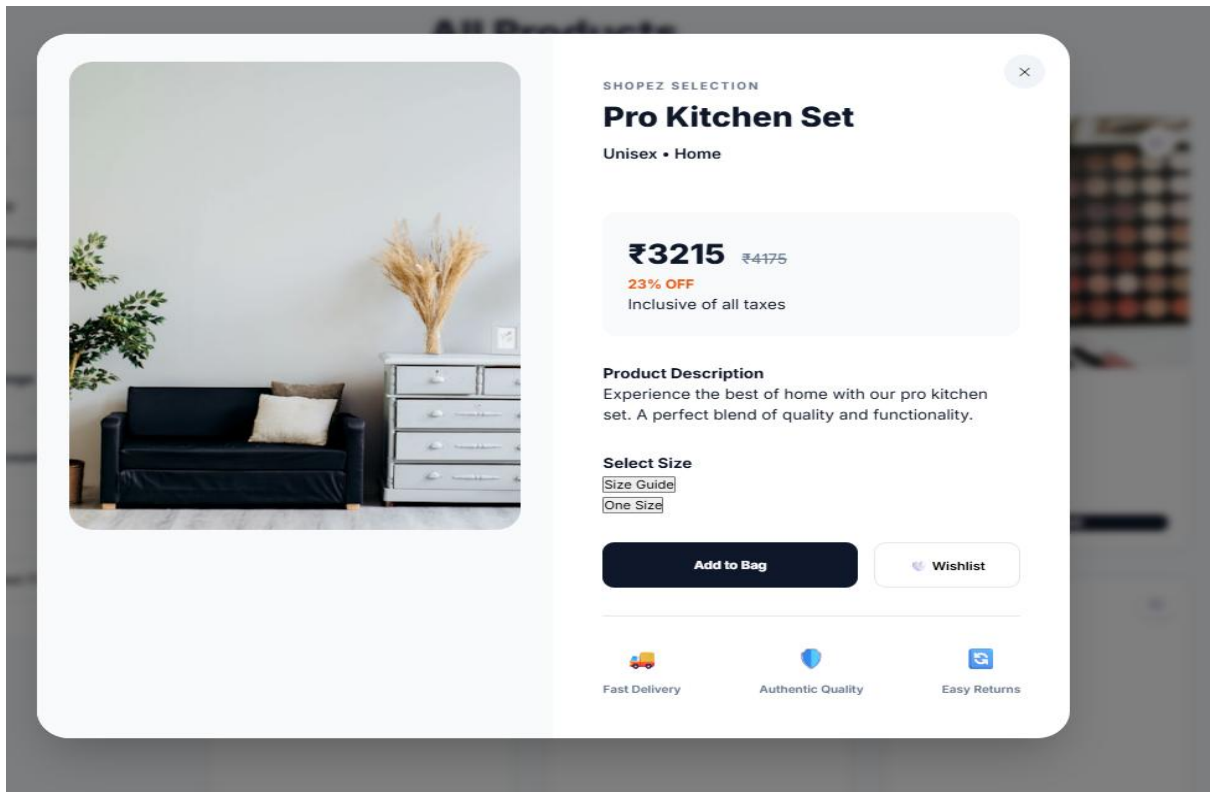
1. Home page



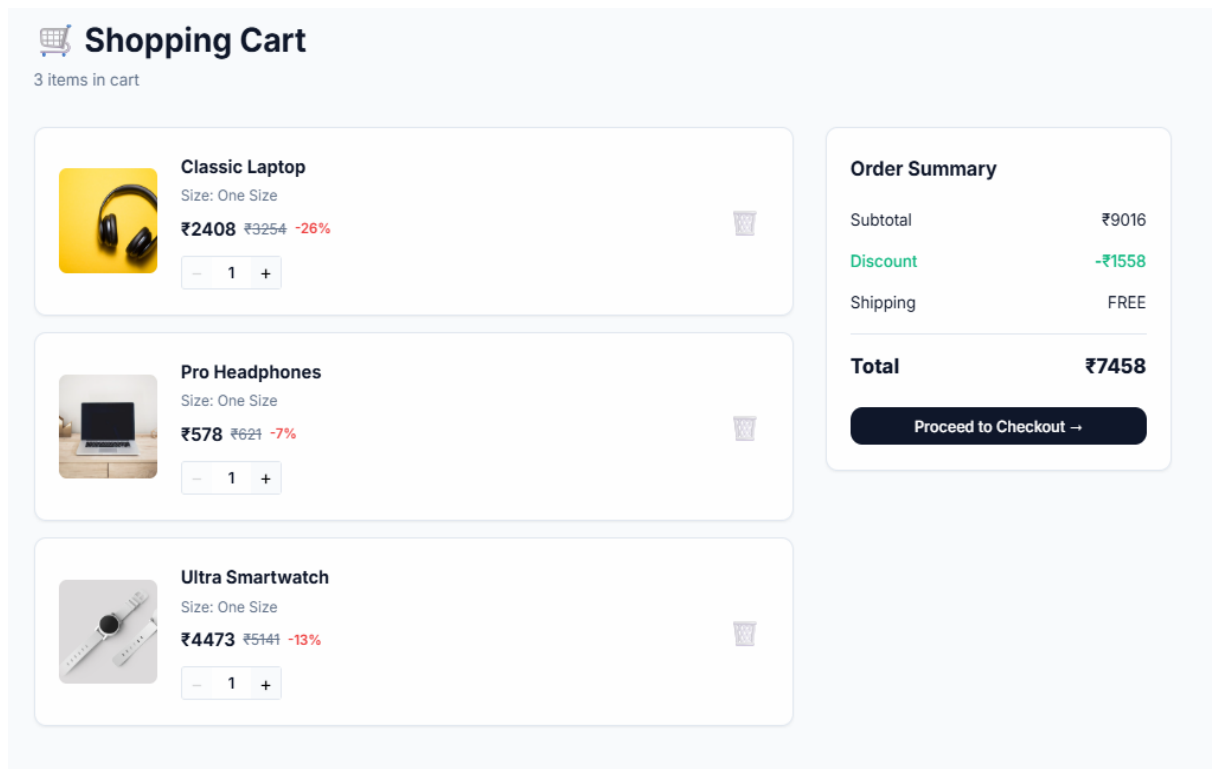
2. Product listing page



3. Product details page



4. Cart page



5. Wishlist page

My Wishlist

2 items saved

-28% OFF



Classic Laptop
Unisex • Electronics
₹2408 ~~₹3254~~
One Size
ADD TO BAG

-7% OFF



Pro Headphones
Unisex • Electronics
₹578 ~~₹621~~
One Size
ADD TO BAG

 **ShopEZ**
Your one-stop destination for trendy fashion and accessories. Shop smart, shop easy with ShopEZ.

Quick Links
[Home](#)
[Products](#)
[Cart](#)
[My Account](#)

Categories
[Men](#)
[Women](#)
[Kids](#)
[Accessories](#)

Contact Us
support@shopez.com
[+91 98765 43210](tel:+919876543210)
[Mumbai, India](#)

6. Checkout page

Checkout

Delivery Details

Full Name

jashwanth

Email

narendulajashwanth007@gmail.com

Mobile Number

+91 XXXXX XXXXX

Delivery Address

Full address with landmark

Pincode

6-digit pincode

Payment Method

Cash on Delivery

Place Order — ₹7458

Order Items (3)

**Classic Laptop**
Size: One Size | Qty: 1
₹2408

**Pro Headphones**
Size: One Size | Qty: 1
₹578

**Ultra Smartwatch**
Size: One Size | Qty: 1
₹4473

Total**₹7458**

 **ShopEZ**
Your one-stop destination for trendy fashion and accessories. Shop smart, shop easy with ShopEZ.

Quick Links
[Home](#)
[Products](#)
[Cart](#)
[My Account](#)

Categories
[Men](#)
[Women](#)
[Kids](#)
[Accessories](#)

Contact Us
support@shopez.com
[+91 98765 43210](tel:+919876543210)
[Mumbai, India](#)

7. Login page



Welcome Back

Login to your ShopEZ account

Username or Email

Password

Login

Don't have an account? [Register here](#)

8. Register page



Create Account

Join ShopEZ for the best shopping experience

Username

Email Address

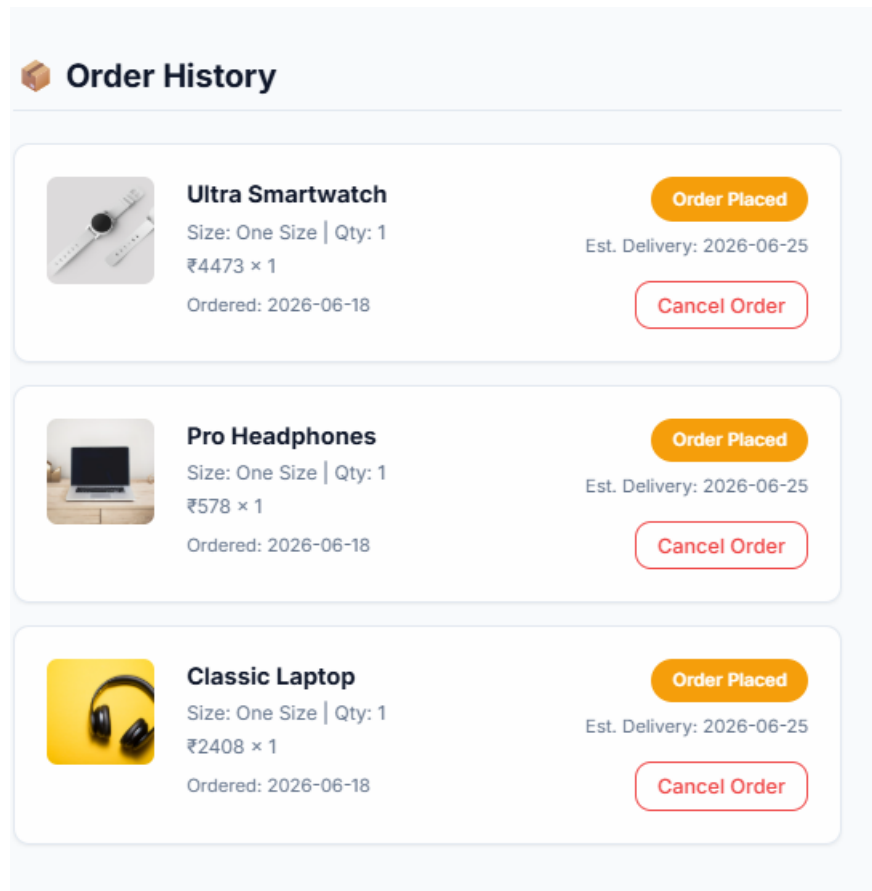
Password

Confirm Password

Create Account

Already have an account? [Login here](#)

9. Orders History page:



Demo Links

Frontend

vip-c2-jashwanth.vercel.app

Backend

<https://vip-c2-jashwanth.onrender.com>

GitHub

<https://github.com/narendulajashwanth007-hash/VIP-C2-Jashwanth.git>

12.Known Issues:

The project is functional and ready for submission. However, the following limitations may be considered:

- Product recommendations are currently basic and not AI-based.
- Delivery tracking is not fully real-time.
- Advanced inventory management is limited.
- Refund processing is stored as metadata but not fully automated with a payment gateway.
- Email notifications depend on proper SMTP configuration.
- Admin analytics can be improved with more detailed charts and reports.

13.Future Enhancements:

The following improvements can be added in the future:

- Real-time order tracking with delivery status updates.
- Advanced product recommendation system.
- AI-based search and personalized product suggestions.
- Seller/vendor dashboard for multiple sellers.
- Invoice generation for completed orders.
- Advanced coupon and promotion engine.
- Product comparison feature.
- Push notifications for order updates.
- Mobile application using React Native.
- Improved analytics with charts and sales reports.
- Return and refund management system.
- Live chat support for customers.
- Google OAuth login.
- Multiple payment gateway support.
- Better stock and warehouse management.